

SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification

1 Paper summary

This paper presents SpecInfer, which is a system to speed up LLM inference using speculative decoding. The main issue is that normal autoregressive decoding generates one token at a time, which limits parallelism and doesn't fully use the GPU. It also creates memory overhead because of the KV cache, especially for long sequences.

Previous approaches try to fix this by using a smaller model to guess future tokens and then verify them with the main LLM. But usually they only guess one sequence, so if the guess is wrong, you don't gain much.

SpecInfer improves this idea by building a tree of possible token sequences instead of just one. So instead of speculating a single path, it considers multiple possible continuations at the same time. Then the main model can verify all of them in parallel in one step. This increases the chance that some of the guessed tokens are correct, even if the small model is not perfect.

They also propose ways to build these trees, either using one model with diverse outputs or combining multiple speculative models. For stochastic decoding, they introduce a method that keeps the output distribution the same as normal decoding, which is important to not affect model quality.

The evaluation shows speedups around 1.5–3x depending on the setup, while keeping the same output behavior. So the main idea is to increase parallelism during decoding without changing the results.

2 Contributions

The first contribution is the idea of tree-based speculative inference. Instead of guessing one sequence, the system builds a tree of candidates, which increases the probability that the correct tokens are included somewhere. This helps improve how many tokens can be processed per step.

The second contribution is how these trees are built. The paper proposes methods to generate diverse candidates, either from one model or multiple ones. This is important because the effectiveness depends on covering enough possible continuations, even if too much diversity could also increase cost.

The third contribution is the verification mechanism and sampling method. The system can verify multiple paths in parallel and still guarantee that the output distribution is correct. This is important because it avoids trading correctness for speed, which is often a problem in similar approaches.

3 Problems

One limitation is that the system depends a lot on the quality of the speculative models. If they are not well aligned with the main LLM, the tree might not include the correct tokens often enough, so the benefit is reduced.

Another issue is the increased complexity. Managing token trees, verification, and correct sampling makes the system more complicated than standard decoding. This could make it harder to implement and maintain, especially in production systems.

A third point is that the speedup depends on a tradeoff. While verifying multiple candidates in parallel is efficient, building and processing the tree also adds overhead. If

the speculative guesses are not accurate enough, the extra work might reduce the gains. So the results depend quite a lot on the scenario.